

15-agile-scrum

15. Agile & Scrum untuk Proyek Tim

Level: □ Beginner

Jam: 6 (3 sesi × 2 JP)

Prasyarat: Git dasar (commit, push, pull), kerja kelompok

Output: Sprint plan + GitHub Project untuk proyek final

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Bedain Agile vs Waterfall dan kapan pakai masing-masing
- Jelaskan 3 peran Scrum (PO, SM, Dev) dan tugasnya
- Jalani 4 upacara Scrum (planning, standup, review, retro)
- Tulis User Story pakai format INVEST
- Estimasi task pakai story point / planning poker
- Setup GitHub Projects buat sprint board
- Baca & interpretasi burndown chart
- Buat Sprint Plan untuk proyek final kelas

Sesi

Sesi	Topik	File
1	Agile & Scrum — Konsep Dasar, Roles, Artifak	01-agile-scrum-basics.md
2	Ceremonies & Backlog Management	02-ceremonies-backlog.md
3	Tools & Penerapan ke Proyek Final	03-tools-real-project.md

Output Akhir Modul

Sprint Plan + GitHub Project — Tim siapkan board, tulis backlog, buat sprint goal, estimasi task, dan setup workflow untuk ngerjain proyek final pakai Scrum.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Buatkan 5 user story untuk aplikasi [nama aplikasi] pakai format 'Sebagai... saya ingin... agar...'"
- "Kasih saya contoh sprint goal yang bagus untuk fitur login, register, dan profile"
- "Evaluasi user story ini: [tempel story]. Apakah sudah INVEST? Kasih saran perbaikan."
- "Simulasikan daily standup: saya jadi Budi, kamu jadi SM. Tanya saya 3 pertanyaan standup."
- "Buatkan template retrospective pake format Start/Stop/Continue untuk tim 4 orang"
- "Bantu saya breakdown user story ini jadi task kecil: [tempel story]"
- "Estimasi dengan planning poker: ceritain cara mainnya terus kasih contoh skenario"

1.1 Agile & Scrum — Konsep Dasar, Roles, Artifak

Agile vs Waterfall

Waterfall	Agile
Rencana semua di awal	Rencana bertahap, bisa berubah
User lihat hasil di akhir	User lihat hasil tiap 1-2 minggu
Error ketemu pas tahap akhir	Error ketemu cepat tiap sprint
Dokumen tebal	Dokumen secukupnya, kode jalan
Cocok untuk proyek besar & pasti	Cocok untuk proyek yang banyak berubah

Waterfall = model air terjun. Langkahnya: Analisis → Desain → Implementasi → Testing → Maintenance. Kalau ada error di desain, ketemunya pas testing — sakitnya di akhir.

Agile = lincah. Kerja dalam **siklus pendek** (sprint). Fitur dikerjain sedikit demi sedikit, langsung demo ke user, langsung perbaiki feedback.

Kapan pakai? Tugas kuliah dengan spek sering berubah → Agile. Proyek akhir dengan dokumen fix dari awal → bisa Waterfall. Tapi rekomendasi: tetap Agile biar adaptif.

Agile Manifesto (4 Nilai)

1. **Individu dan interaksi** → daripada proses dan alat
2. **Perangkat lunak yang berfungsi** → daripada dokumentasi lengkap
3. **Kolaborasi dengan pelanggan** → daripada negosiasi kontrak
4. **Merespons perubahan** → daripada mengikuti rencana

Intinya: Kode jalan > dokumen tebal. Ngobrol dengan user > bikin spek 50 halaman.

Scrum — Kerangka Kerja Agile

Scrum adalah kerangka kerja Agile paling populer. Bukan metode kaku — tapi **mainan yang tinggal pakai**.

3 Pilar Scrum

1. **Transparency** — Semua orang tau apa yang dikerjakan
 2. **Inspection** — Sering cek hasil kerja
 3. **Adaptation** — Sesuaikan rencana kalau melenceng
-

3 Peran Scrum (Roles)

Product Owner (PO) — "Suara User"

Tugas:

- Tulis **User Story** (kebutuhan fitur dari sisi user)
- Urutkan prioritas di **Product Backlog**
- Jelaskan "kenapa" fitur ini penting
- Terima / tolak hasil sprint

Di kelas: Satu orang jadi PO. Bisa dosen atau siswa yang paling ngerti kebutuhan proyek. PO bukan manager — PO tidak suruh-suruh tim cara ngerjain.

Scrum Master (SM) — "Pelindung Tim"

Tugas:

- Pastikan Scrum jalan benar
- Hapus hambatan (blocker) tim
- Fasilitasi meeting (standup, planning, review, retro)

- Bukan ketua tim — bukan bos

Di kelas: SM bisa bergilir tiap sprint. SM tidak perlu jago coding. Cukup peduli sama proses tim.

Developer (Dev) — "Tukang Kode"

Tugas:

- Memilih task sendiri (self-assign)
- Ngerjain task → selesai di standup
- Tes kode sendiri
- Bertanggung jawab bareng

Di kelas: Semua anggota tim sisanya. Semua Dev setara — tidak ada "programmer senior" dan "anak gambar".

Aturan emas: PO bilang "APA", SM jagain "BAGAIMANA", Dev kerjain "siapa yang ngapain".

3 Artifak Scrum

Product Backlog

Daftar semua fitur yang **mungkin** dikerjakan. Hidup — terus berubah. Diurutkan dari paling penting ke paling tidak penting.

Format:

[PRIORITAS] [TIPE] [JUDUL]
P1 - FEATURE - Login Google
P2 - FEATURE - Export PDF
P3 - BUG - Tombol simpan double click error

User Story adalah unit backlog:

Sebagai [pengguna], saya ingin [fitur] agar [alasan].

Contoh:

Sebagai siswa, saya ingin login pakai Google agar tidak perlu daftar ulang.

Sprint Backlog

Task-task dari Product Backlog yang **diambil untuk sprint ini**. Di-breakdown jadi subtask ~4-16 jam.

Contoh breakdown:

FEATURE: Login Google

- [] [4 jam] Setup OAuth Google Console
- [] [6 jam] Buat halaman login (tombol Google)
- [] [2 jam] Test login dengan akun dummy

Increment

Hasil sprint yang **selesai dan bisa dipakai**. Tiap sprint harus ngasih increment yang berfungsi — walau cuma 1 halaman.

Syarat Increment: Kode sudah di-review, sudah di-merge ke branch utama, sudah jalan di lokal.

Siklus Sprint (Mermaid)

```
graph TD
    PB[Product Backlog] --> SP[Sprint Planning]
    SP --> SB[Sprint Backlog]
    SB --> DEV[Developer bekerja<br/>Daily Standup tiap hari]
    DEV --> INCR[Increment]
    INCR --> SR[Sprint Review]
    SR --> RETRO[Sprint Retrospective]
    RETRO --> PB
    SR --> PB
    RETRO --> PB

    style PB fill:#elf5fe,stroke:#01579b
    style SP fill:#fff3e0,stroke:#e65100
    style SB fill:#e8f5e9,stroke:#1b5e20
    style DEV fill:#f3e5f5,stroke:#4a148c
    style INCR fill:#elf5fe,stroke:#01579b
    style SR fill:#fff3e0,stroke:#e65100
    style RETRO fill:#fce4ec,stroke:#b71c1c
```

Penjelasan diagram:

1. **Product Backlog** — Semua fitur yang mungkin
 2. **Sprint Planning** — Tim pilih fitur untuk sprint ini
 3. **Sprint Backlog** — Task yang dikerjakan selama sprint
 4. **Developer + Daily Standup** — Proses pengerjaan, tiap hari cek progress
 5. **Increment** — Hasil sprint yang bisa dipakai
 6. **Sprint Review** — Demo ke PO, PO kasih feedback
 7. **Sprint Retrospective** — Evaluasi proses tim
-

Contoh Mapping Peran (4 Orang)

Nama	Peran	Tugas Utama
Budi	Product Owner	Tulis backlog, urut prioritas, terima/tolak fitur
Ani	Scrum Master	Fasilitasi meeting, hapus blocker
Caca	Developer	Coding fitur
Dedi	Developer	Coding fitur

Catatan: Di tim 3 orang, PO bisa merangkap Dev (tapi tidak ideal). SM bisa bergilir tiap sprint.

Latihan

Latihan 1: Bedain Waterfall vs Agile (Diskusi Kelompok — 10 menit)

Baca skenario berikut, tentukan cocok pakai Waterfall atau Agile, dan jelaskan alasannya.

Skenario	Cocok Pakai?	Kenapa?
Proyek akhir SMK — spek udah fix dari awal		
Aplikasi startup — fitur bisa berubah tiap minggu		
Sistem ATM bank — keamanan ketat, jarang update		
Tugas kelompok 2 minggu — dosen suka ganti permintaan		

Latihan 2: Tentukan Peran (Diskusi — 10 menit)

Untuk tiap aktivitas, siapa yang paling bertanggung jawab? (PO / SM / Dev)

Aktivitas	Peran
Nulis user story "Sebagai user..."	
Nentuin prioritas fitur	
Ngebut deadline terdesak	
Ngasih tahu dosen kalau progress terhambat	
Nulis kode fitur login	
Fasilitasi daily standup	
Nolak fitur karena tidak sesuai kebutuhan	

Latihan 3: Bikin Product Backlog Awal (Kelompok — 15 menit)

Tentukan proyek final kalian (aplikasi yang akan dibuat). Buat minimal **6 user story** di Product Backlog dengan format:

P[prioritas] - [TIPE] - [Judul]

Sebagai [pengguna], saya ingin [fitur] agar [alasan].

Contoh untuk aplikasi perpustakaan sekolah:

P1 - FEATURE - Login anggota

Sebagai anggota perpustakaan, saya ingin login pakai NIS/NIP agar bisa akses fitur p

Latihan 4: Gambar Siklus Sprint (Individu — 10 menit)

Gambar ulang diagram siklus Sprint di kertas atau digital. Tambahkan:

- Durasi tiap fase (misal: planning 2 jam, sprint 2 minggu, review 1 jam)
- Siapa yang hadir di tiap fase

Ringkasan: Agile = lincah, Scrum = framework Agile paling populer. Ada 3 peran (PO, SM, Dev), 3 artifak (Product Backlog, Sprint Backlog, Increment). Siklus sprint: planning → kerja → review → retro → ulang.

1.2 Ceremonies & Backlog Management

4 Upacara Scrum (Ceremonies)

1. Sprint Planning — Awal Sprint (Maks 2 jam)

Tujuan: Tentukan apa yang dikerjakan sprint ini dan bagaimana cara menyelesaikannya.

Agenda:

1. PO jelaskan prioritas Product Backlog (top 3-5)
2. Tim diskusi & klarifikasi
3. Tim estimasi effort (story point atau jam)
4. Tim pilih task yang masuk Sprint Backlog
5. Breakdown task jadi subtask (masing-masing ≤ 16 jam)
6. Tulis **Sprint Goal** — 1 kalimat tujuan sprint

Output: Sprint Backlog + Sprint Goal.

Sprint Goal contoh: "Sprint ini kita buat fitur login, registrasi, dan landing page agar user bisa daftar dan lihat halaman depan."

2. Daily Standup — Tiap Hari (15 menit, berdiri)

Tiap orang jawab 3 pertanyaan:

1. **Apa yang saya kerjakan kemarin?**
2. **Apa yang akan saya kerjakan hari ini?**
3. **Apa hambatan saya?**

Aturan:

- Berdiri — biar singkat
- Bukan laporan ke SM — komitmen ke tim
- Jangan bahas solusi di standup. Bahas **setelah** standup (scrumming session)
- Kalau ada blocker, SM catat dan bantu setelah standup

3. Sprint Review — Akhir Sprint (Maks 1 jam)

Tujuan: Demo hasil kerja ke PO dan stakeholder.

Agenda:

1. Tim demo increment ke PO / dosen (langsung buka aplikasi, bukan slide)
2. PO cek apakah sesuai ekspektasi
3. PO terima atau tolak fitur
4. Diskusi: apa yang mau diubah?
5. Product Backlog diperbarui

Bukan presentasi slide — langsung demo kode jalan.

4. Sprint Retrospective — Setelah Review (Maks 45 menit)

Tujuan: Tim evaluasi proses — bukan orang.

Format Start-Stop-Continue:

Kategori	Arti	Contoh
Start	Hal baru yang perlu mulai dilakukan	"Mulai bikin branch sendiri sebelum coding"
Stop	Kebiasaan buruk yang harus dihentikan	"Stop commit langsung ke main"
Continue	Hal baik yang perlu dipertahankan	"Lanjutin daily standup tiap pagi"

Output: 1-2 aksi konkret untuk sprint berikutnya.

Golden rule retro: Blame tidak ada. Semua bicara tentang **proses**, bukan orang.

Mermaid: Alur Satu Sprint Penuh

```

timeline
    title Satu Sprint (2 Minggu)
    Hari 1 : Sprint Planning (2 jam)
        Pilih backlog
        Estimasi task
        Tulis Sprint Goal
    Hari 2-9 : Daily Standup (15 menit)
        Developer coding
        PR review
        Merge fitur
    Hari 10 : Sprint Review (1 jam)
        Demo ke P0
        Sprint Retrospective (45 menit)
        Start / Stop / Continue

```

User Stories — Bahasa Kebutuhan

User story adalah format standar untuk menulis kebutuhan fitur. Bukan dokumen teknis — ditulis dari **sisi pengguna**.

Format Standar

Sebagai [peran pengguna], saya ingin [fitur] agar [manfaat/nilai bisnis].

Contoh:

Sebagai siswa, saya ingin melihat daftar buku yang tersedia agar tahu buku apa yang k

INVEST — Kriteria User Story yang Baik

Kriteria	Arti	Contoh ☐
Independent	Bisa dikerjakan sendiri, tidak dependen story lain	"Login"
Negotiable	Bisa didiskusikan, bukan instruksi kaku	"Tampilkan daftar buku"
Valuable	Memberi nilai ke pengguna	"Export data ke PDF"
Estimable	Bisa diestimasi (tidak terlalu abstrak)	"Filter buku berdasarkan kata"
Small	Ukuran pas untuk 1 sprint	"Halaman profile user"

Kriteria	Arti	Contoh □
Testable	Jelas kapan selesai — bisa di-test	"Tombol simpan disabled kala

Estimasi Task

Story Point

Bukan jam — **ukuran relatif** seberapa kompleks suatu task.

Skala umum (Fibonacci):

1, 2, 3, 5, 8, 13, 21

Story Point	Arti	Contoh
1	Sangat kecil, tinggal ngetik	"Ganti warna tombol"
2	Kecil, sudah jelas caranya	"Tambah validasi email"
3	Sedang, perlu sedikit riset	"Buat halaman login"
5	Besar, ada beberapa bagian	"Integrasi API pembayaran"
8	Sangat besar, perlu breakdown	"Buat sistem notifikasi"
13+	Terlalu besar — harus dipecah	"Bikin aplikasi dari nol"

Planning Poker — Cara Estimasi Tim

1. Tiap dev pegang kartu (1, 2, 3, 5, 8, 13)
2. PO bacakan 1 user story
3. Tim diskusi sebentar
4. Hitung mundur — semua tunjuk kartu bersamaan
5. Kalau ada selisih jauh (1 vs 8), diskusi kenapa
6. Voting ulang sampai konsensus

Sprint: Sesi 1 (30 menit)

Story: "Sebagai siswa, saya bisa login pakai Google"

Voting ronde 1:

Budi: 3 Ani: 5 Caca: 5 Dedi: 3

Selisih kecil → ambil rata-rata: 4 → pilih 5

→ Story Login Google: 5 story point

Estimasi Timeline per Sprint

Rumus sederhana:

Jam tersedia per minggu × jumlah dev × 2 minggu = total jam sprint
Contoh: 10 jam × 4 dev × 2 minggu = 80 jam per sprint

Dari 80 jam, kurangi **20% buffer** (meeting, istirahat, revisi) → **64 jam efektif**.

Backlog Refinement (Grooming)

Aktivitas rutin — **bukan upacara wajib** tapi sangat disarankan — untuk merapikan Product Backlog.

Kapan: Antar sprint atau tengah sprint, 1 jam.

Aktivitas:

1. Hapus story yang sudah tidak relevan
 2. Split story yang terlalu besar (epic → stories)
 3. Tambah detail ke story yang belum jelas
 4. Re-prioritaskan sesuai feedback terbaru
 5. Split story 13+ → beberapa story 3-5
-

Contoh Sprint Backlog Lengkap

Untuk tim 4 orang, sprint 2 minggu:

Story	Story Point	Assignee	Status
Login Google	5	Budi	☐ Done
Halaman Register	5	Ani	☐ Done
Landing Page	3	Caca	☐ In Progress
Profile User	3	Dedi	☐ To Do
Total	16		

Latihan

Latihan 1: Tulis User Story (Individu — 10 menit)

Tulis ulang kebutuhan berikut jadi **user story** format yang benar.

Kebutuhan	User Story
User bisa cari buku berdasarkan judul	
Admin bisa tambah buku baru	
User bisa lihat riwayat peminjaman	
Sistem kirim notifikasi kalau buku harus dikembalikan	

Latihan 2: Evaluasi User Story dengan INVEST (Kelompok — 10 menit)

Evaluasi story berikut. Mana yang sudah INVEST? Mana yang perlu diperbaiki?

User Story	INVEST?	Perlu
"Sebagai admin, saya ingin sistem yang cepat"		
"Buat fungsi hitung denda"		
"Sebagai siswa, saya ingin login pakai email agar bisa akses fitur peminjaman"		
"Bikin semua fitur aplikasi perpustakaan"		

Latihan 3: Planning Poker Simulasi (Kelompok — 15 menit)

Ambil 3 user story dari Product Backlog yang dibuat di sesi 1. Lakukan simulasi planning poker:

1. Setiap anggota voting (diam-diam) story point untuk tiap story
2. Tunjuk barengan
3. Diskusi kalau ada selisih jauh (≥ 3 poin)
4. Voting ulang sampai konsensus
5. Catat hasil estimasi

Story	Voting 1	Voting 2	Final
[story 1]			
[story 2]			
[story 3]			

Latihan 4: Sprint Retro Simulation (Kelompok — 10 menit)

Bayangkan kalian sudah menyelesaikan 1 sprint untuk proyek RPL. Isi tabel Start-Stop-Continue:

Kategori	Isi
Start (mulai lakukan)	

Kategori	Isi
Start	
Stop (hentikan)	
Stop	
Continue (pertahankan)	
Continue	

Aksi konkret: Pilih 1 dari Start dan 1 dari Stop → tulis langkah konkret untuk sprint depan.

Ringkasan: 4 upacara Scrum = Planning, Standup, Review, Retro. User story pakai format INVEST. Estimasi pakai story point (1-21) + planning poker. Backlog refinement jaga backlog tetap rapi.

1.3 Tools & Penerapan ke Proyek Final

GitHub Projects — Sprint Board Digital

GitHub Projects adalah fitur bawaan GitHub untuk manage task pakai **board visual**.

Cara Setup GitHub Projects

1. Buka repo proyek kalian di GitHub
2. Klik tab **Projects** → **Create project**
3. Pilih **Board** (kanban style)
4. Beri nama: Sprint Board - [Nama Proyek]
5. Klik **Create**

Struktur Kolom Standar

Kolom	Arti
☐ To Do	Task yang sudah direncanakan tapi belum dikerjakan
☐ In Progress	Task yang sedang dikerjakan seseorang
☐ Review	Task selesai coding, tunggu PR di-review
☐ Done	Task sudah selesai + di-merge

Drag & Drop Workflow

To Do → In Progress → Review → Done

Sehari-hari:

1. Ambil task dari **To Do** → drag ke **In Progress**
 2. Assign ke diri sendiri
 3. Coding + commit
 4. Bikin PR → drag ke **Review** + tag reviewer
 5. Review selesai, merge → drag ke **Done**
-

Mermaid: Alur Task di Sprint Board

```
graph LR
    TD[□ To Do] -->|Ambil task| IP[□ In Progress]
    IP -->|Selesai coding, buat PR| RV[□ Review]
    RV -->|PR di-approve & merge| DN[□ Done]
    RV -->|PR ditolak, perlu revisi| IP

    style TD fill:#e3f2fd,stroke:#1565c0
    style IP fill:#fff3e0,stroke:#e65100
    style RV fill:#f3e5f5,stroke:#6a1b9a
    style DN fill:#e8f5e9,stroke:#2e7d32
```

Milestones di GitHub

Milestones = **sprint** di GitHub timeline.

Cara Setup Milestone

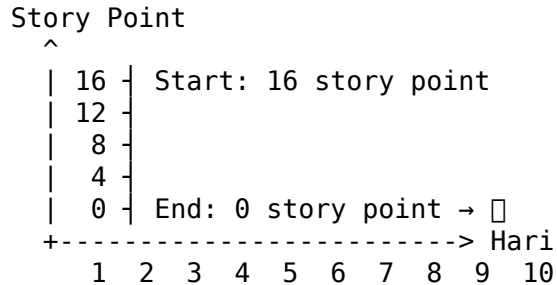
1. Repo → **Issues** → **Milestones** → **Create a milestone**
2. Isi:
 - **Title:** Sprint 1
 - **Due date:** tanggal akhir sprint
 - **Description:** Sprint Goal (contoh: "Fitur auth + landing page")
3. Setelah buat, tiap issue bisa di-assign ke milestone ini

Melihat progress: GitHub otomatis hitung berapa % issue yang closed vs open di milestone.

Burndown Chart

Burndown chart = grafik yang menunjukkan sisa effort (story point) tiap hari selama sprint.

Cara Baca



Garis biru (ideal): Penurunan linear — tiap hari selesai ~1.6 story point. **Garis oranye (aktual):** Realita tim — naik-turun.

Burndown Simulation (Mermaid)

```
---
title: Burndown Chart – Sprint 1
---
xyChart
  x-axis "Hari" [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  y-axis "Sisa Story Point" 0 --> 20
  line "Ideal" [16, 14.4, 12.8, 11.2, 9.6, 8.0, 6.4, 4.8, 3.2, 1.6]
  line "Aktual" [16, 16, 14, 14, 11, 9, 9, 6, 3, 0]
```

Arti:

Hari	Sisa SP	Arti
1	16	Sprint start, semua task belum disentuh
2-3	16→14	Mulai ngerjain, belum selesai apa-apa (kaget)
4-5	14→11	Progress mulai jalan
6-7	11→9	Stuck — ada blocker (PR belum di-review)
8-9	9→3	Buru-buru ngejar deadline
10	0	□ Selesai tepat waktu

Kapan Perlu Khawatir?

Kondisi	Arti	Tindakan
Garis aktual jauh di atas ideal	Tim kurang progress	Cek blocker, kurangi scope
Garis aktual di bawah ideal	Tim lebih cepat	Boleh ambil task tambahan
Garis aktual flat 3 hari berturut	Ada masalah serius	SM wajib intervensi

Menerapkan Scrum ke Proyek Final

Langkah-langkah Setup Sprint 1

□ H-1: Persiapan

- ☐ Tentukan anggota tim & peran (PO, SM, Dev)
- ☐ Buat repo GitHub (1 repo untuk 1 tim)
- ☐ Setup GitHub Project Board
- ☐ Buat milestone "Sprint 1" (due date: 2 minggu)

□ Hari 1: Sprint Planning

- ☐ PO siapkan Product Backlog (minimal 6 user story)
- ☐ Tim estimasi pake planning poker
- ☐ Pilih story untuk Sprint Backlog (total ~16-20 story point)
- ☐ Tulis Sprint Goal
- ☐ Breakdown task di GitHub Issues
- ☐ Assign task ke anggota tim

Contoh Sprint Goal:

"Sprint 1: User bisa login, daftar, dan lihat halaman utama aplikasi."

□ Hari 2-9: Eksekusi Pagi (15 menit): Daily Standup

Tiap orang:

1. "Kemarin saya ____"
2. "Hari ini saya ____"
3. "Hambatan: ____"

Sepanjang hari:

- Coding di branch feature/*
- Commit pakai convention (feat, fix, docs, dll)
- Bikin PR ke develop
- Review PR teman (≤ 24 jam)
- Update status di GitHub Project board

□ Hari 10: Review & Retro

- Sprint Review — demo ke PO/dosen
 - PO tandai story yang selesai □ / ditolak □
 - Sprint Retrospective — isi Start/Stop/Continue
 - Update Product Backlog untuk Sprint 2
 - Tutup milestone Sprint 1
-

Mermaid: Alur Lengkap Sprint 1

flowchart TD

```
A[Sprint Planning<br/>Pilih backlog + estimasi] --> B[Daily Standup<br/>Tiap hari]
B --> C[Coding & Review]
C --> D[PR siap]
D --> E[Review oleh teman]
E --> F[Approved]
F --> G[Merge ke develop]
G --> H[Revisi]
H --> I[Perbaiki & push ulang]
I --> J[D]
J --> K{Ada task lain?}
K -- Ya --> B
K -- Tidak --> L[Sprint Review<br/>Demo ke PO]
L --> M[Sprint Retrospective<br/>Start / Stop / Continue]
M --> N[Sprint 2 siap □]
```

```
style A fill:#e1f5fe,stroke:#01579b
style B fill:#f3e5f5,stroke:#4a148c
style H fill:#fff3e0,stroke:#e65100
style I fill:#fce4ec,stroke:#b71c1c
style J fill:#e8f5e9,stroke:#1b5e20
```

Checklist Proyek Final — Sprint 1

Gunakan checklist ini untuk memastikan tim siap.

Sebelum Sprint Mulai

- Semua anggota punya akses ke repo GitHub
- GitHub Project Board sudah jadi (4 kolom)
- Milestone "Sprint 1" sudah dibuat
- Product Backlog minimal 6 user story
- Sprint Goal sudah ditulis

Selama Sprint

- ☐ Daily standup tiap hari (di chat atau lisan)
- ☐ Tiap task ada di board → status selalu update
- ☐ PR selalu di-review ≤ 24 jam
- ☐ Tidak ada commit langsung ke main/develop
- ☐ Convention commit dipakai

Akhir Sprint

- ☐ Sprint Review dijadwalkan dengan P0/dosen
 - ☐ Semua task di board sudah di Done atau dikembalikan ke backlog
 - ☐ Retro dijalankan → output aksi konkret
 - ☐ Milestone Sprint 1 di-closes
-

Latihan

Latihan 1: Setup GitHub Project (Praktik — 15 menit)

Di repo proyek kalian:

1. Buat GitHub Project Board (Board template)
2. Tambah 4 kolom: **To Do, In Progress, Review, Done**
3. Buat milestone "**Sprint 1**" — due date 2 minggu dari sekarang
4. Buat 6 **issues** dari Product Backlog sesi 1
5. Assign tiap issue ke milestone Sprint 1
6. Pindahkan 2 issue ke **To Do**

Screenshot hasil dan kumpulkan ke dosen.

Latihan 2: Simulasi Burndown (Kelompok — 10 menit)

Tim kalian punya 20 story point di Sprint 1. Berikut data progress harian:

Hari	Sisa Story Point
1	20
2	20
3	18
4	18
5	15
6	12
7	12
8	8

Hari	Sisa Story Point
9	4
10	0

Pertanyaan:

1. Apakah tim ini selesai tepat waktu? □ / □
2. Di hari berapa tim paling produktif?
3. Ada hari flat (tidak ada progress)? Hari berapa?
4. Apa yang sebaiknya SM lakukan di hari flat?

Latihan 3: Rencanakan Sprint 1 untuk Proyek Kalian (Kelompok — 20 menit)

Buat dokumen **Sprint Plan** di Google Docs / Notion / Markdown dengan isi:

Sprint Plan — [Nama Proyek]

Info Sprint

- Sprint: 1
- Durasi: [tanggal mulai] — [tanggal selesai]
- Tim: [nama-nama anggota]

Sprint Goal

[1 kalimat — apa yang akan dicapai sprint ini]

Sprint Backlog

Issue	Story Point	Assignee	Status
-----	-----	-----	-----
To Do			
To Do			
To Do			
Total			

Peran

- PO: [nama]
- SM: [nama]
- Dev: [nama], [nama]

Latihan 4: Refleksi — Siap Pakai Scrum? (Individu — 5 menit)

Isi jurnal refleksi:

1. Apa hal paling menarik tentang Scrum yang kamu pelajari?

2. Apa yang paling bikin khawatir saat pakai Scrum untuk proyek final?
3. Menurutmu, apa tantangan terbesar tim kalian saat pakai Scrum?
4. Satu hal yang kamu usahakan sendiri untuk sukses di Sprint 1:

Ringkasan: GitHub Projects = sprint board digital. 4 kolom: To Do, In Progress, Review, Done. Milestones = sprint. Burndown chart = tracker progress harian. Setup Sprint 1: planning → eksekusi (standup tiap hari) → review → retro. Checklist proyek final siap dipakai!